

诺亚方舟（诺亚保管库）项目技术概览

版本：v1.0.38 | 最后更新：2026-04-30 | 项目组：诺亚方舟

一、项目简介

诺亚保管库是一套基于多模态生物识别技术的身份认证终端系统，部署在专用硬件设备上，用于对进出人员进行高安全级别的身份验证。系统支持密码、胁迫码、虹膜、掌纹四种认证方式的任意组合，并与上级管理平台（IAMS）通过 MQTT 协议实时联动。

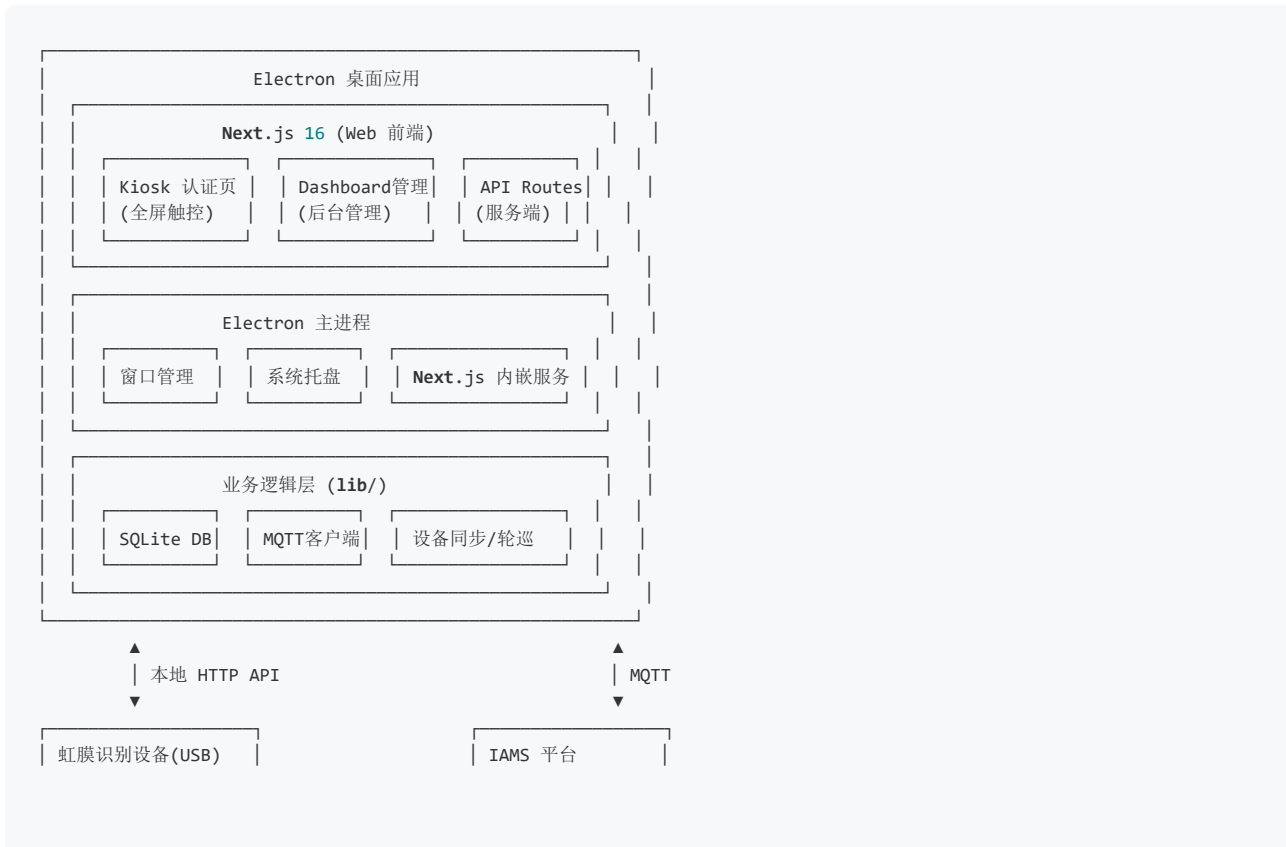
核心场景

- 组合认证**：人员可按预设顺序依次完成密码→虹膜→掌纹等多步认证，全部通过后方可通行
- 单独认证**：也支持单一认证方式独立使用
- 凭证远程管理**：IAMS 通过 MQTT 下发凭证添加/更新/删除指令，设备自动同步到虹膜和掌纹硬件
- 通行记录上报**：认证成功后记录通行日志并上报给 IAMS
- 设备状态心跳**：每 10 秒向 IAMS 发送心跳，报告设备在线状态

典型用户

- 现场使用人员**：通过触控屏完成身份认证
- 管理员**：通过密码进入后台管理系统，管理设备、凭证、查看通行记录和日志
- IAMS 平台运维**：通过 MQTT 远程管理凭证和设备

二、技术架构



掌纹识别设备(USB)

(远程 MQTT Broker)

架构特点

- **Electron + Next.js 融合**: Electron 作为壳, 内嵌 Next.js 作为 Web 框架, 同时提供前端 UI 和后端 API
- **本地 SQLite 数据库**: 所有凭证、通行记录、设备配置、系统设置均存储在本地
- **MQTT 实时通信**: 与 IAMS 平台通过 MQTT 协议双向通信, 接收凭证指令、上报状态和通行记录
- **设备直连**: 虹膜和掌纹识别设备通过本地 HTTP API 直接连接, 不经过网络

三、技术栈详情

3.1 前端

技术	版本	用途
Next.js	16.2.1	Web 框架, App Router 模式, 提供页面和 API Routes
React	19.2.4	UI 组件库
TypeScript	5.x	全栈类型安全
Tailwind CSS	3.4.1	原子化 CSS, 响应式设计
PostCSS	8.x	CSS 处理

3.2 桌面端

技术	版本	用途
Electron	33.4.11	桌面应用容器, 全屏 Kiosk 模式
electron-builder	25.1.8	打包为 Windows NSIS 安装程序

3.3 后端 / 数据层

技术	版本	用途
@libsql/client	0.17.2	SQLite 数据库驱动
mqtt	5.15.0	MQTT 客户端, 连接 IAMS Broker
aedes	1.0.1	本地 MQTT Broker (开发测试用)
crypto (Node.js 内置)	—	AES 加密/解密凭证内容

3.4 开发工具

技术	版本	用途
ESLint	9.x	代码检查

技术	版本	用途
Turbopack	(内置)	Next.js 构建工具

3.5 运行环境

组件	说明
操作系统	Windows 10/11 (x64)
Node.js	v24 (由 electron-builder 打包嵌入)
数据库	SQLite (本地文件 <code>noah-ark.db</code>)
安装包格式	NSIS 安装程序 (诺亚保管库 <code>Setup X.X.X.exe</code>)

四、核心功能模块

4.1 认证终端 (Kiosk)

页面	路径	功能
待机页	<code>/</code>	系统待机，点击进入认证
身份确认页	<code>/kiosk</code>	输入身份编码（支持数字+字母虚拟键盘）
认证方式选择	<code>/kiosk/select</code>	根据凭证类型展示可用认证方式
密码认证	<code>/kiosk/password</code>	密码输入 + 虚拟数字键盘
虹膜认证	<code>/kiosk/iris</code>	虹膜设备轮询等待识别结果
掌纹认证	<code>/kiosk/palm</code>	掌纹设备轮询等待识别结果
组合认证	<code>/kiosk/combined</code>	多步组合认证流程（密码→虹膜→掌纹）
认证成功页	<code>/kiosk/success</code>	显示认证结果和人员信息

4.2 管理后台 (Dashboard)

页面	路径	功能
设备管理	<code>/dashboard/devices</code>	虹膜/掌纹设备状态监控、配置管理
凭证管理	<code>/dashboard/credentials</code>	凭证增删改查、设备同步状态
MQTT 指令	<code>/dashboard/mqtt-events</code>	IAMS 指令接收与响应记录
通行记录	<code>/dashboard/pass-logs</code>	认证通行日志查询
系统设置	<code>/dashboard/settings</code>	MQTT 配置、设备配置、管理员密码、测试工具
服务器日志	<code>/dashboard/logs</code>	心跳数据查看（排查设备ID冲突）

4.3 API 接口

分类	端点	功能
认证	/api/auth/verify-identity	验证身份编码
认证	/api/auth/verify-password	验证密码凭证
认证	/api/auth/verify-palm	掌纹设备验证
认证	/api/auth/verify-iris	虹膜设备验证
认证	/api/auth/types	获取可用认证类型
认证	/api/auth/admin-password	管理员密码验证
认证	/api/auth/decrypt	解密凭证内容
认证	/api/auth/settings	认证相关设置
凭证	/api/credentials	凭证 CRUD
设备	/api/devices	设备信息查询
设备	/api/devices/sync	设备同步控制
设备	/api/devices/iris-lock	虹膜设备锁定/解锁
设备	/api/device/palm/query	掌纹设备查询
设备	/api/device/iris/verify	虹膜设备验证
设备	/api/device/iris/records	虹膜记录查询
通行记录	/api/pass-logs	通行记录查询
通行记录	/api/pass-log/upload	通行记录上报
MQTT	/api/mqtt-events	MQTT 事件查询
设置	/api/settings	系统设置 CRUD
设置	/api/logs/heartbeat	心跳数据查询
日志	/api/combined-auth-log	组合认证日志
测试	/api/test/credentials-batch	凭证批量操作（测试用）
其他	/api/version	应用版本号
其他	/api/health	健康检查
其他	/api/init	系统初始化
其他	/api/client-config	客户端配置

4.4 核心库模块

模块	文件	功能
数据库	lib/database.ts	SQLite 连接和表管理

模块	文件	功能
凭证管理	lib/db-credentials.ts	凭证 CRUD、查询
MQTT 客户端	lib/mqtt-client.ts	连接 IAMS、接收指令、上报状态
设备同步	lib/device-sync.ts	凭证同步到虹膜/掌纹设备
设备轮巡	lib/device-poller.ts	定时检查设备在线状态
同步队列	lib/sync-queue.ts	消息队列管理
同步调度	lib/sync-scheduler.ts	同步任务调度
设置管理	lib/settings.ts	系统设置读写 (settings.json)
设备属性	lib/db-device-attrs.ts	设备属性存储
通行记录	lib/db-pass-logs.ts	通行记录数据库操作
胁迫码	lib/db-duress.ts	胁迫码处理
用户管理	lib/db-users.ts	用户信息管理
加密	lib/crypto.ts	AES 加解密
上传	lib/upload-pass-log.ts	通行记录上报 IAMS

五、关键业务流程

5.1 凭证下发流程

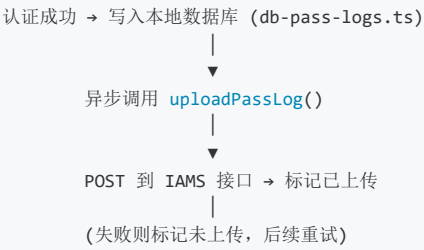


5.2 组合认证流程



▼
写入通行记录 → 上报 IAMS → 显示成功页

5.3 通行记录上报流程



六、数据模型

6.1 核心数据表

credentials (凭证表)

字段	类型	说明
credential_id	TEXT PK	凭证 ID
person_id	TEXT	人员编码
person_name	TEXT	人员姓名
type	INTEGER	凭证类型 (5=密码, 7=虹膜, 8=掌纹, 9=胁迫码)
content	TEXT	凭证内容 (加密存储, 掌纹为 userId:featureData)
custom_id	TEXT	自定义 ID (掌纹设备 userId)
auth_type_list	TEXT	认证类型列表 (逗号分隔)
auth_model	INTEGER	认证模式 (1=单凭证, 820=组合认证)
enable	INTEGER	是否启用

pass_logs (通行记录表)

字段	类型	说明
id	TEXT PK	记录 ID
person_id	TEXT	人员编码
auth_type	TEXT	认证方式描述
auth_types	TEXT	认证类型列表 (JSON 数组)
credential_id	TEXT	凭证 ID
created_at	TEXT	创建时间 (北京时间)

字段	类型	说明
uploaded	INTEGER	是否已上报 IAMS

settings（系统设置表）

字段	类型	说明
key	TEXT PK	设置键名
value	TEXT	设置值（JSON 字符串）

6.2 凭证类型编码

编码	类型	说明
5	密码	数字密码，虚拟键盘输入
7	虹膜	虹膜设备识别
8	掌纹	掌纹设备识别
9	胁迫码	胁迫状态下使用的密码

七、部署与打包

7.1 开发模式

```
npm run dev          # 启动 Next.js（端口 3001）
npm run electron     # 启动 Electron
npm run mqtt:broker  # 启动本地 MQTT Broker（测试用）
```

7.2 生产打包

```
cd electron && npm run dist
# 输出: release/ 诺亚保管库 Setup X.X.X.exe
```

7.3 打包后目录结构

```
安装目录/
├── 诺亚保管库.exe          # Electron 主程序
├── resources/
│   ├── app/
│   │   ├── .next/          # Next.js 构建产物
│   │   ├── app/            # Next.js 页面文件
│   │   ├── lib/            # 后端逻辑模块
│   │   ├── node_modules/   # 依赖
│   │   └── package.json
│   └── app.asar            #（如启用）
└── node.exe               # 嵌入式 Node.js 运行时
```

7.4 数据存储位置

数据类型	路径
数据库	%APPDATA%/noah-ark-electron/noah-ark.db
系统设置	%APPDATA%/noah-ark-electron/settings.json
客户端配置	%APPDATA%/noah-ark-electron/client-config.json
日志文件	%APPDATA%/noah-ark-electron/logs/
IAMS 载荷	%APPDATA%/noah-ark-electron/iams_payloads/

八、外部系统集成

8.1 IAMS 平台

协议	方向	用途
MQTT	双向	凭证指令下发、状态上报、通行记录上报
HTTP	出向	通行记录上传、心跳发送

8.2 虹膜识别设备

接口	方法	用途
/memberSave	POST	上传人员信息到设备
/members	GET	查询设备人员列表
/lock	POST	锁定设备（写保护）
/unlock	POST	解锁设备
/deviceStatus	GET	设备状态查询

8.3 掌纹识别设备

接口	方法	用途
110	POST	下发用户特征数据
109	POST	获取用户特征数据
103	POST	设备识别（掌纹比对）
102	POST	删除用户特征
105	GET	设备状态查询

九、安全设计

- 1. **AES 加密存储**: 密码凭证内容使用 AES 加密后存入数据库
- 2. **胁迫码机制**: 胁迫码触发时静默记录通行，表面显示认证成功
- 3. **管理员密码保护**: 后台管理需通过密码验证，支持密码修改
- 4. **本地数据库**: 所有数据存储在本地，不依赖外部数据库服务
- 5. **MQTT 认证**: 连接 IAMS Broker 使用用户名密码认证

十、项目文件结构

```
noah-ark-v2/
├── app/                                # Next.js App Router
│   ├── api/                            # API Routes (25+ 端点)
│   │   ├── auth/                       # 认证相关 API
│   │   ├── credentials/                # 凭证管理 API
│   │   ├── devices/                   # 设备管理 API
│   │   ├── device/                    # 设备直连 API
│   │   ├── pass-logs/                 # 通行记录 API
│   │   ├── settings/                  # 系统设置 API
│   │   ├── mqtt-events/               # MQTT 事件 API
│   │   ├── logs/                      # 日志 API
│   │   └── test/                      # 测试 API
│   │   └── ...
│   ├── kiosk/                          # 认证终端页面
│   │   ├── combined/                  # 组合认证
│   │   ├── iris/                      # 虹膜认证
│   │   ├── palm/                      # 掌纹认证
│   │   ├── password/                  # 密码认证
│   │   ├── select/                    # 认证方式选择
│   │   └── success/                    # 认证成功页
│   ├── dashboard/                     # 管理后台页面
│   │   ├── devices/                   # 设备管理
│   │   ├── credentials/               # 凭证管理
│   │   ├── mqtt-events/               # MQTT 指令
│   │   ├── pass-logs/                 # 通行记录
│   │   ├── settings/                  # 系统设置
│   │   └── logs/                      # 服务器日志
│   ├── fonts/                          # 字体文件
│   ├── layout.tsx                      # 根布局
│   ├── page.tsx                        # 首页 (待机页)
│   └── globals.css                     # 全局样式
├── components/                          # React 组件
│   └── kiosk/                          # Kiosk 专用组件
│       ├── Footer.tsx                 # 底部状态栏 + 管理员入口
│       └── IdleTimer.tsx               # 无操作倒计时组件
├── lib/                                # 后端逻辑模块
│   ├── database.ts                     # SQLite 数据库
│   ├── db-credentials.ts               # 凭证 CRUD
│   ├── db-pass-logs.ts                 # 通行记录
│   ├── db-device-attrs.ts              # 设备属性
│   ├── db-duress.ts                    # 胁迫码
│   ├── db-users.ts                     # 用户管理
│   ├── mqtt-client.ts                  # MQTT 客户端
│   ├── device-sync.ts                  # 设备同步
│   ├── device-poller.ts                # 设备轮巡
│   ├── sync-queue.ts                   # 同步队列
│   ├── sync-scheduler.ts               # 同步调度
│   ├── settings.ts                     # 系统设置
│   ├── crypto.ts                       # AES 加解密
│   ├── upload-pass-log.ts              # 通行记录上传
│   └── init.ts                          # 系统初始化
├── electron/                           # Electron 主进程
└── main.ts                             # 主进程入口
```

```
|   |─ package.json          # Electron 配置
|   └─ build/              # 打包配置
├─ docs/                  # 项目文档
|   └─ 诺亚方舟项目/
|       └─ 功能实现文档.md
|       └─ 版本更新日志.md
├─ data/                  # 运行时数据目录
├─ logs/                  # 日志目录
└─ scripts/              # 工具脚本
    └─ mqtt-broker.js     # 本地 MQTT Broker
    └─ mqtt-test-client.ts # 测试 MQTT 客户端
```